Conditional Random Fields

Probabilistic Models for Segmenting and Labeling Sequence Data

Lafferty, McCallum, Pereira (2001)

Linear-chain CRF — implemented from scratch

Christopher Bradshaw

Problem and Paper Overview

The setting

- **Sequence labeling**: assign a label to every token in a sequence.
- Example tasks: POS tagging, NER, shallow parsing, **language identification**.
- The structure matters — a token's best label depends on its neighbors.

What the paper contributes

- A **discriminative** alternative to HMMs.
- Directly models $p(y \mid x)$ — no generative story for x.
- Fixes the **label-bias problem** that plagues MEMMs.
- Globally normalized → every path competes against every other path.

> "CRFs combine the per-state expressiveness of MEMMs with the global normalization of HMMs."

Method Summary

Score of a labeling

$$s(\mathbf{y}, \mathbf{x}) = \sum_i \left[\lambda \cdot f(y_{i-1}, y_i, \mathbf{x}, i) \right]$$

- **Emission** features tie labels to observations.
- **Transition** features tie labels to their predecessor.
- Plus **start** / **stop** boundary scores.

Conditional probability

$$p(\mathbf{y}|\mathbf{x}) = \frac{\exp\ s(\mathbf{y}, \mathbf{x})}{Z(\mathbf{x})}$$

Three key algorithms

Inference **Viterbi** — argmax over label sequences via dynamic programming.

Training **Forward-backward** — compute Z(x) and marginals in log-space.

Learning **Algorithm S** — iterative scaling update:

```
weight[k] += (1/S) · log(observed[k] / expected[k])
```

Per feature: if gold fires it more than the model expects, push the weight up. Until they match.

Implementation Details

What I built from scratch

- **Log-space forward-backward** (with `scipy.logsumexp` for stability).
- **Viterbi** decoding with backpointers.
- **Algorithm S** training loop with monotonic-log-likelihood check.
- Unary, pairwise, start, and stop **marginals** for expected counts.
- An **MEMM baseline** sharing the same features — locally normalized, decoded with Viterbi.

Feature templates (per token)

- `word`, `prefix3`, `suffix2/3`
- `is_cap`, `is_all_caps`, `has_digit`
- `is_twitter_specific` (@mentions, #hashtags, URLs)
- `prev_word`, `next_word`

Experiment Setup

Dataset: LinCE `LID_spaeng`

Spanish-English code-switched **tweets**, token-level language IDs.

Split	Sentences	Tokens
Train	21,030	253,221
Validation	3,332	40,391
Test	8,289	97,341

Label distribution (train)

Label	Share
`lang2` (es)	44.6%
`lang1` (en)	31.8%
`other`	21.4%
`ne`	2.1%
`ambiguous`, `mixed`, `fw`, `unk`	< 0.3% combined

Heavily imbalanced — the rare classes barely exist.

Main Results

CRF on validation

Metric	Value	What it measures
Token accuracy	0.9599	fraction of tokens labeled correctly
Weighted F1	0.9521	per-class F1, weighted by support
Macro F1	0.3879	per-class F1, unweighted
Switch-point acc	0.8916	accuracy at positions where gold label changes

50 iterations · ~3,696s (61 min 36 sec)

Per-class F1 (validation)

Label	F1	Support
`lang1`	0.96	16,712
`lang2`	0.96	14,955
`other`	0.98	7,830
`ne`	0.19	815
rare (ambig/mixed/fw/unk)	0.00	< 80

Full Results Dump

MEMM Baseline Results

Analysis and Failure Cases

Where it works

- **Dominant classes** (lang1/lang2/other): 96-98% F1 — strong signal from word, suffix, and twitter-specific features.
- **Switch points**: 89.2% accuracy, clear evidence the transition weights are pulling their weight.
- **Training stability**: log-likelihood monotonically ↑ across all 200 iterations — an end-to-end correctness signal for forward-backward + Algorithm S.

Where it struggles

- **Named entities (`ne`)**: F1 improves to 0.19, but recall is still only 0.11 — most named entities are still absorbed by `lang1` / `lang2`.
- **Rare classes**: `ambiguous`, `mixed`, `fw`, `unk` — F1 = 0. Class imbalance overwhelms Algorithm S: unregularized weights can't overcome the prior.
- **Training cost**: 62 min for 50 passes over 21k sentences. Algorithm S is an $O(N \cdot R^2)$ hit per iteration — feasible here, but L-BFGS is much faster.

Conclusion

What I learned

- **The math is simpler than the reputation.** Linear-chain CRF is two DPs (Viterbi, forward-backward) and a counting update rule.
- **Global normalization is elegant.** The optimizer still matters.
- **Algorithm S is beautifully self-checking.** Monotonic log-likelihood = ~100% of correctness debugging.

Was the paper's claim supported?

Partly The CRF handles structured sequence labeling and uses context, but the MEMM baseline wins on this feature set.

Caveat The training story (Algorithm S, unregularized) is the weakest link — optimization dominates the theory here.

0.9599 CRF VALIDATION TOKEN ACCURACY · LINCE LID SPAENG